

tension accelerates while tracking common peril of dropping into a loophole that would derive a maximum performance.

Best practices that ensure the rendering and painting codes generator by the framework are totally efficient

- **Controlling build cost:** Minimisation of build cost is a target because repetitive and costly build methods come into play while rebuilding the predecessor Widgets.
- **Splitting up of Widgets:** Instead of using a large, a single widget that uses a large build function; a widget can be split into components that differ on encapsulation.

The descendant widgets get reconstructed when set slate is brought forth on the slate. When similar event of a child widget is re-encountered, the traversal to rebuild the child widgets terminate.

Applying effects only when required

Applying effects only when required saves on the additional costs that might be incurred while doing so because the effects are generally extremely expensive. Some of them employ an even more expensive operation called Savelayer. The Savelayer option calls forth many expensive actions by allocating an offscreen buffer which might render target switches.

There are a few general rules that should be borne in mind while applying effects.

The usage of the Opacity usage should be minimised by the employment of transparent section and the usage of Opacity API page from therein.



- Clipping action doesn't call forth Savelayer but it is still very expensive and hence clipping should be explicitly used only when required and so that no additional cost will be incurred.
- Shadermask, Color filter, Chip, Text are these widgets which employ Savelayer and hence should be avoided because they would prove extremely expensive.
- Save Layer can also be potentially avoided by using a Fade image option